

UD5 - Alta disponibilidad

Diseño de servicios redundantes, tolerantes a fallos y recuperables.

Datos de la unidad	Información
Módulo	Seguridad y Alta Disponibilidad
Curso	2.º ASIR
Modalidad	Semipresencial
Duración	14 horas

Índice

1. [Fundamentos de alta disponibilidad](#)
2. [Redundancia de infraestructura y red](#)
3. [Clústeres y failover en Linux](#)
4. [Balanceo, IP virtual y continuidad de red](#)
5. [Virtualización, contenedores y diseño HA](#)
6. [Resumen](#)
7. [Recursos](#)
8. [Relación con los resultados de aprendizaje](#)

1. Fundamentos de alta disponibilidad

1.1. Introducción

La alta disponibilidad, o HA, busca que un servicio permanezca operativo y accesible el mayor tiempo posible, reduciendo las interrupciones no planificadas. Es necesaria cuando una caída afecta de forma relevante a usuarios, ingresos, seguridad, obligaciones legales o continuidad de una organización.

HA no equivale a eliminar todos los fallos. Combina prevención, detección, redundancia, conmutación automática y recuperación para reducir el tiempo de indisponibilidad. También se relaciona con continuidad de negocio y recuperación ante desastres: la primera mantiene las operaciones y la segunda permite recuperarlas tras un incidente grave.

1.2. Objetivos

Al finalizar la unidad, el alumnado será capaz de:

- Diferenciar disponibilidad, redundancia, tolerancia a fallos y continuidad.
- Identificar puntos únicos de fallo en una infraestructura.
- Interpretar MTBF, MTTR, RPO y RTO en un diseño HA.
- Seleccionar mecanismos de redundancia de hardware, almacenamiento y red.
- Explicar el funcionamiento de clústeres, quórum, *fencing* y *failover*.
- Diseñar servicios con balanceo de carga, IP virtual y comprobaciones de salud.

- Valorar el papel de la virtualización, los contenedores y la automatización.

1.3. Disponibilidad y puntos únicos de fallo

La disponibilidad expresa la proporción de tiempo en que un servicio funciona correctamente:

$$D = \frac{MTBF}{MTBF + MTTR}$$

El MTBF es el tiempo medio entre fallos y el MTTR el tiempo medio de reparación. Los llamados "nueves" traducen un porcentaje anual en tiempo de caída aproximado: 99.9% permite unas 8.76 horas, 99.99% unos 52.56 minutos y 99.999% unos 5.26 minutos.

Un punto único de fallo, o SPOF, es un componente cuya caída interrumpe un servicio. Puede ser un firewall, un switch, una fuente de alimentación, un host de virtualización, un enlace o una base de datos. El primer paso de un diseño HA es identificarlos y decidir cuáles deben eliminarse según el impacto y el coste.

1.4. Redundancia, tolerancia y objetivos de recuperación

La redundancia incorpora componentes adicionales; la tolerancia a fallos permite mantener el servicio cuando uno falla; el *failover* traslada un recurso a un componente disponible. Los modelos activo-pasivo mantienen un nodo de reserva, mientras que los activo-activo reparten la carga entre varios nodos.

RPO establece la pérdida máxima de datos aceptable y RTO el tiempo objetivo de recuperación. Por ejemplo, un RPO de 15 minutos puede exigir copias o replicación frecuentes, mientras que un RTO de una hora requiere procedimientos ensayados y recursos listos para recuperar. RAID y copias de seguridad se estudian en UD2: RAID mejora la disponibilidad frente a fallos de disco, pero no sustituye las copias ni la recuperación ante borrados o ransomware.

2. Redundancia de infraestructura y red

2.1. Hardware, alimentación y almacenamiento

La redundancia puede aplicarse a servidores, fuentes de alimentación, ventiladores, controladoras, interfaces de red y almacenamiento. Los componentes redundantes deben conectarse, cuando sea posible, a rutas eléctricas y de red diferentes; duplicar un equipo que depende del mismo switch o SAI no elimina todos los riesgos.

Un SAI proporciona autonomía limitada y tiempo para un apagado controlado; un generador puede sostener cortes prolongados. La potencia, autonomía, prioridades y procedimiento de apagado deben dimensionarse y probarse. NAS, SAN, replicación y sistemas distribuidos permiten reducir riesgos de almacenamiento, pero exigen diseñar coherencia de datos y recuperación.

2.2. Redundancia de red y agregación de enlaces

Una red resiliente incorpora rutas y equipos alternativos para evitar que un único enlace, switch, router o firewall interrumpa el servicio. Las topologías de malla, anillo o núcleo-distribución-acceso pueden ofrecer varios caminos si se diseñan y supervisan correctamente.

La agregación de enlaces combina varias interfaces físicas en un canal lógico. LACP, normalizado en IEEE 802.3ad, permite aumentar la capacidad agregada y mantener conectividad si falla uno de los enlaces. Debe

configurarse de forma compatible en ambos extremos y no garantiza que una única conexión individual use todo el ancho de banda del grupo.

Los enlaces redundantes pueden crear bucles. STP y sus variantes RSTP o MSTP bloquean rutas según la topología y habilitan alternativas cuando detectan un fallo. La segmentación, los protocolos de enrutamiento y la monitorización completan el diseño de una red disponible.

3. Clústeres y failover en Linux

3.1. Componentes y modelos de clúster

Un clúster HA reúne varios nodos para ofrecer un servicio de forma coordinada. Sus componentes habituales son nodos, red de comunicación, almacenamiento compartido o replicado, recursos gestionados, comprobaciones de salud y mecanismos para evitar operaciones simultáneas no seguras.

En un diseño activo-pasivo, un nodo ejecuta el servicio y otro está preparado para asumirlo. En activo-activo, varios nodos atienden peticiones y normalmente se utiliza balanceo. La elección depende de si la aplicación permite ejecutarse de forma concurrente, cómo mantiene el estado y qué consistencia requieren sus datos.

3.2. Corosync, Pacemaker, quórum y fencing

Corosync proporciona comunicación entre nodos, detección de pertenencia y quórum. Pacemaker utiliza esa información para gestionar recursos como servicios, sistemas de archivos o direcciones IP, y decide dónde deben ejecutarse según las restricciones definidas.

El quórum evita que una parte aislada del clúster tome decisiones críticas sin mayoría suficiente. El *split-brain* ocurre cuando nodos o grupos aislados creen poder gestionar el mismo recurso, con riesgo de corrupción de datos. El *fencing* aísla de forma fiable un nodo que no responde, por ejemplo apagándolo o bloqueando su acceso al almacenamiento, antes de mover un recurso crítico a otro nodo.

3.3. Datos, comprobaciones y conmutación

Un clúster necesita decidir dónde están los datos y cómo preservar su consistencia. Puede usar almacenamiento compartido, replicación de bloque como DRBD, sistemas distribuidos como GlusterFS o mecanismos propios de una base de datos. No existe una solución universal: se debe valorar latencia, integridad, comportamiento ante particiones y recuperación.

Las comprobaciones de salud deben verificar tanto que el proceso está activo como que el servicio responde correctamente. Cuando se detecta un fallo, el clúster aplica el procedimiento de *failover*: detiene o aísla el recurso si es necesario, activa el destino y confirma que el servicio funciona antes de anunciarlo a clientes.

4. Balanceo, IP virtual y continuidad de red

4.1. Balanceadores y proxy inverso

Un balanceador de carga distribuye peticiones entre varios servidores disponibles para mejorar disponibilidad, rendimiento y escalabilidad. Los algoritmos habituales incluyen *round-robin*, menor número de conexiones, reparto ponderado y hash de IP. Este último puede aportar afinidad de sesión, pero reduce flexibilidad si la distribución de clientes es desigual.

HAProxy y Nginx pueden actuar como balanceadores y proxy inverso. El proxy inverso oculta los servidores internos, puede terminar TLS, aplicar filtrado y almacenar contenido en caché. El propio balanceador también puede ser un SPOF, por lo que los servicios críticos suelen desplegarlo de forma redundante.

4.2. IP virtual y VRRP

VRRP ofrece una puerta de enlace o dirección IP virtual compartida por varios routers o servidores. Un miembro actúa como *master* y los demás como *backup*; si los respaldos dejan de recibir anuncios, uno asume la IP virtual. Los clientes mantienen la misma puerta de enlace y no necesitan reconfigurarse.

Keepalived implementa VRRP en Linux y puede asociar la IP virtual a comprobaciones de salud. Se utiliza tanto para redundancia de gateways como para mantener disponible una IP de balanceo. HSRP y GLBP son alternativas propietarias habituales en dispositivos Cisco.

4.3. Diseño y pruebas

La disponibilidad real se valida mediante pruebas controladas: caída de un nodo, fallo de un enlace, indisponibilidad de un backend, reinicio planificado y restauración de datos. Las pruebas deben tener alcance, horario, criterios de parada y plan de reversión. Una solución HA no está completa hasta que se comprueban sus alertas, tiempos de conmutación y comportamiento en recuperación.

5. Virtualización, contenedores y diseño HA

5.1. Virtualización y contenedores

La virtualización facilita aislar servicios, crear recursos rápidamente, migrar máquinas y reiniciar cargas en otros hosts. Plataformas como Proxmox, VMware o Hyper-V pueden incorporar clústeres y automatización de recuperación. Para que una máquina virtual sea realmente disponible, deben considerarse también los hosts, redes y almacenamiento de los que depende.

Los contenedores reducen la sobrecarga al compartir el sistema operativo del host. Orquestadores como Kubernetes gestionan réplicas, comprobaciones de salud, despliegues y reprogramación de cargas. Aun así, la disponibilidad de una aplicación depende de sus datos, configuración, secretos, red y diseño de estado, no solo del número de réplicas.

5.2. Planificación de una arquitectura HA

Una propuesta HA parte de servicios críticos, objetivos RPO/RTO, dependencias y presupuesto. Debe identificar SPOF, elegir controles proporcionados y definir quién responde a alertas. Añadir tecnología sin conocer el servicio puede incrementar el riesgo operativo.

Un servicio web de ejemplo puede usar dos servidores detrás de un proxy inverso, dos balanceadores con una IP virtual, almacenamiento o base de datos replicados, copias de seguridad independientes, monitorización centralizada y procedimientos de recuperación documentados. La arquitectura debe poder crecer, mantenerse y probarse sin interrumpir innecesariamente el servicio.

6. Resumen

La alta disponibilidad combina redundancia, detección, automatización y procedimientos de recuperación para reducir indisponibilidades. El diseño debe eliminar SPOF relevantes, mantener la consistencia de los

datos y validar el comportamiento ante fallos reales o simulados.

Clústeres, balanceadores, VRRP, LACP, virtualización y almacenamiento replicado son herramientas, no objetivos por sí mismos. Su valor depende de que respondan a requisitos medibles de disponibilidad, RPO y RTO.

7. Recursos

- [Pacemaker](#)
- [Corosync](#)
- [Keepalived](#)
- [HAProxy](#)
- [Documentación de LACP](#)
- [Proxmox VE](#)
- [Kubernetes: high availability](#)

8. Relación con los resultados de aprendizaje

Esta unidad contribuye principalmente al **RA6**, mediante el diseño e implantación de soluciones de alta disponibilidad con redundancia, virtualización, almacenamiento, clústeres, balanceo y recuperación.

También se relaciona con el **RA2**, por la monitorización y detección de fallos, y con el **RA4**, cuando las soluciones HA se integran con dispositivos y controles perimetrales.